C-Grade Engine

Ubuntu Setup & Troubleshooting Guide

Step-by-step terminal commands for preparing a new Linux system

Purpose: Use this guide to set up and launch C-Grade Engine on a fresh Ubuntu computer.

Project assumptions: C-only grading, Streamlit UI, GCC/Cppcheck/Lizard analysis, and optional Valgrind, perf, and local Ollama for AI-generated tests.

Important: The exact dependency list can only be guaranteed by checking the project's current requirements.txt and imports. This guide covers the tools identified in the project context and common setup failures.

Quick plan

1. Copy or clone the project.
2. Install Ubuntu system tools.
3. Create and activate a project-local .venv.
4. Install Python dependencies.
5. Set up Ollama and the correct model tag if AI test generation is enabled.
6. Verify the tools and launch Streamlit.

1. Before starting

Open Terminal (Ctrl+Alt+T). You need internet for package installation and, if applicable, the Ollama model. Copy the complete project folder, including analyzer files, rules/, tests/, submissions/, configuration files, and requirements.txt if present.

If using Git, replace the placeholders with your actual repository URL and folder name:

```
git clone <YOUR_REPOSITORY_URL>
cd <YOUR_PROJECT_FOLDER>
```

If copying files manually, enter the folder containing ui.py and confirm:

```
pwd
ls -la
find . -maxdepth 2 -name 'ui.py' -o -name 'requirements.txt'
```

Do not type the angle-bracket placeholders literally.

2. Install Ubuntu system packages

These are system tools, not pip packages. Run:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-full python3-pip build-essential gcc cppcheck valgrind
```

Tool	Purpose
python3, python3-venv, python3-full	Python and virtual environments
build-essential, gcc	Compiler/build tools for C
cppcheck	Static analysis for C
valgrind	Memory diagnostics if enabled

C-Grade Engine accepts C only. Use gcc with the C standard configured by the project (for example, -std=c11). g++ is not required for C-only grading.

3. Create and activate .venv

Run these commands from the project root—the folder containing ui.py. Recreate the virtual environment on the new machine instead of copying a .venv from another computer.

```
pwd
ls
python3 -m venv .venv
source .venv/bin/activate
which python
python --version
python -m pip --version
```

Expected: which python should point to a path ending in /.venv/bin/python, not /usr/bin/python. The pip path should also be inside .venv.

Whenever you open a new terminal, activate the environment again:

```
cd <YOUR_PROJECT_FOLDER>
source .venv/bin/activate
```

4. Install Python dependencies

If requirements.txt exists, use it as the preferred dependency list:

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

If the project does not have requirements.txt, install the currently identified baseline packages:

```
python -m pip install --upgrade pip
```

```
python -m pip install streamlit lizard psutil
```

Only install additional packages when your code imports them. If you get ModuleNotFoundError: No module named 'NAME', activate .venv and install the corresponding package using python -m pip install NAME. Import names and package names can differ; check the repository or ask for help with the exact error.

Do not use sudo pip install or --break-system-packages. They bypass system Python protection and are not needed inside .venv.

5. Verify GCC and analysis tools

```
gcc --version
cppcheck --version
valgrind --version
python -c "import streamlit, lizard, psutil; print('Python packages OK')"
which gcc
which cppcheck
which valgrind
```

If a tool is missing, install it using apt, for example:

```
sudo apt update
sudo apt install -y gcc cppcheck valgrind
```

If the project uses perf for complexity measurement, check it:

```
perf --version
sudo apt install linux-tools-common
```

The perf package and kernel permissions vary by Ubuntu version. If perf remains unavailable, check whether the estimator can be configured to skip it; do not assume this analyzer can run without perf unless its configuration supports that.

6. Ollama setup (only if AI test generation is enabled)

Ollama is a separate application, not a Python package. Check whether it is installed:

```
ollama --version
ollama list
```

If the command is missing, install Ollama using the official Linux instructions: https://ollama.com/download/linux. Then check whether the service is running. On some setups you can start it with:

```
ollama serve
```

If it reports that the address is already in use, the service may already be running. Open another terminal and run ollama list.

Your project notes mention qwen3.5-2b. The model identifier in the Python code must match the exact tag installed in Ollama. Check ollama list first. If the configured model is missing, pull the exact supported tag. For example, use this only if that tag is supported by your Ollama setup:

```
ollama pull qwen3.5:2b
ollama list
```

Do not assume qwen3.5-2b and qwen3.5:2b are interchangeable. If generation times out, confirm the service is running, the model is downloaded, and the timeout in the runner is sufficient.

7. Launch the app

```
# From the project root, with .venv active
python -m streamlit run ui.py
```

Streamlit should print a local URL, commonly http://localhost:8501. Open it in the browser on the same computer. Keep the terminal open. To stop the app, press Ctrl+C in that terminal.

If the port is busy, try another port:

```
python -m streamlit run ui.py --server.port 8502
```

Always prefer python -m streamlit run ui.py so the app uses the active virtual environment's Python interpreter.

8. Common problems and fixes

externally-managed-environment

Likely cause: System pip is being used.

Fix: Run source .venv/bin/activate; check which python; then install with python -m pip install PACKAGE. The Python path must contain .venv/bin/python.

No module named 'lizard' or another module

Likely cause: The package is missing from the environment running the app.

Fix: Activate .venv, install the correct package with python -m pip install PACKAGE, and relaunch with python -m streamlit run ui.py.

No such file: .venv/bin/activate

Likely cause: Wrong folder or virtual environment not created.

Fix: Use pwd and ls, enter the project root, then run python3 -m venv .venv.

gcc/cppcheck/valgrind not found

Likely cause: System tool is not installed.

Fix: Run sudo apt update and sudo apt install PACKAGE for the missing tool.

Ollama connection refused or timeout

Likely cause: Service is stopped, model is missing, or generation is slow.

Fix: Check ollama list and service status; verify model tag; test generation before running the full grading workflow.

Model not found

Likely cause: The model name in code differs from the installed tag.

Fix: Compare the configured name with ollama list, then pull the exact supported tag.

Permission denied

Likely cause: A project file or folder lacks access permissions.

Fix: Check ls -l and folder ownership. Avoid running the whole app with sudo.

Streamlit opens but analyzer fails

Likely cause: Tool, path, configuration, or service is missing.

Fix: Read the full terminal traceback; verify oracle.c/test paths and run each required tool's --version command.

Port 8501 already in use

Likely cause: Another Streamlit process is using the port.

Fix: Stop the other process with Ctrl+C, or use --server.port 8502.

perf unavailable or permission denied

Likely cause: Kernel package/support or performance-counter permissions vary.

Fix: Check the Ubuntu/kernel setup and analyzer configuration; skip perf only if the project supports that.

9. Final checklist

■ Project files are present and ui.py is in the project root.

■ The terminal shows (.venv), and which python points inside .venv.

- requirements.txt is installed, or baseline Python packages are installed.
- gcc and cppcheck work; Valgrind/perf work if enabled by the project.
- Ollama is running and the model is available if AI test generation is enabled.
- The model tag in code matches the output of ollama list.
- oracle.c and configured test directories exist.
- The app starts with python -m streamlit run ui.py.
- A small sample C submission has been evaluated before batch grading.

10. Quick commands

```
# New terminal session
cd <YOUR_PROJECT_FOLDER>
source .venv/bin/activate

# Check Python environment
which python
python -m pip --version

# Install project requirements
python -m pip install -r requirements.txt

# Start C-Grade Engine
python -m streamlit run ui.py

# Stop Streamlit: press Ctrl+C
```

If setup fails, save the complete error and the outputs of python --version, which python, python -m pip --version, gcc --version, cppcheck --version, and ollama list (if used). This information will make troubleshooting much faster.

Scope note: this guide is based on the project architecture and tools currently known. Verify the final dependency list against the actual repository's requirements.txt and imports; optional analyzers may need extra packages or system tools.